"Joint optimization of energy consumption and time delay in IoT-fog-cloud computing environments using NSGA-II metaheuristic algorithm".

https://link.springer.com/article/10.1007/s12652-021-03388-2

Class of the paper: IoT Networks, Fog/Edge Computing, Multi-Objective Optimization.

Method: Metaheuristic evolutionary optimization.

Approach: Formulating the routing decision as a multi-objective mathematical problem and solving it via genetic algorithms enhanced with differential evolution to balance power vs. speed.

The backbone of the work: A three-tier IoT-Fog-Cloud architecture utilizing NSGA-II hybridized with Minimax Differential Evolution (MMDE) for offloading decisions.

1. What is the main problem they were trying to solve?

Finding the optimal routing assignments for computation offloading in IoT environments. Sending tasks to the cloud causes too much delay, but local/edge processing drains energy. Finding the exact optimal trade-off is an NP-hard problem that scales poorly as the network grows.

2. Their contributions

- Modeled the offloading problem without relying on traditional, often inaccurate queuing theory assumptions (like Poisson distribution).
- Formulated a highly comprehensive pathway model that includes direct IoT-to-Cloud, IoT-to-Fog, and Fog-to-Cloud offloading.
- Successfully applied the MMDE (Minimax Differential Evolution) technique to NSGA-II to solve the multi-objective problem, achieving near-perfect routing decisions in under 2 seconds.

3. Each methodology structure

- **Formulation:** Defined the objective function as $\min(w_1 \cdot \tilde{E} + w_2 \cdot \tilde{t})$; subject to memory and deadline constraints.
- **Baseline (NIP):** Solved the mathematical matrix for small scales to establish a ground truth.
- **Evolutionary Search (NSGA-II):** Encoded decisions into arrays (chromosomes). Used tournament crossover and mutation to generate child routing scenarios, ranking them via non-dominated sorting and crowding distance.
- **Optimization (MMDE):** Integrated a min-heap structure and bottom-boosting to intercept weak offspring from the genetic algorithm and force them into better fitness areas before the next generation cycle.

4. Real-time? If yes, then what are the constraints, or limitations?

- **Almost Yes**, the NSGA-II-MMDE approach operates in near real-time, converging on an optimal routing map in less than 2 seconds for up to 2500 IoT requests.
- **Limitations:** The standard mathematical approach (NIP) completely fails in real-time due to scaling complexity. Additionally, the Bees Algorithm (BA) proved to have too high

an energy variance and lower accuracy, making it a poor choice for this specific architecture.

5. The future work, and any potential room to improve or optimize their work.

- **Authors' Future Work:** They suggest integrating Reinforcement Learning or clustering methods alongside these metaheuristics. They also plan to apply this exact NSGA-II-MMDE logic to process-heavy medical image fusion and chaotic color image encryption.
- **Room to Improve:** Genetic algorithms, even fast ones, still require recalculating populations when the environment changes. A significant optimization would be entirely replacing the real-time execution of the NSGA-II algorithm with a trained Deep Reinforcement Learning (DRL) agent (like a Deep Q-Network). The DRL could use the NSGA-II outputs purely as offline training data, allowing the live system to infer offloading decisions in **milliseconds** rather than 2 seconds.

Notations :

Knapsack is the *type* of puzzle, and NP-hard is the *difficulty level* of that puzzle.

The Knapsack Problem Mapping

- **The Knapsack:** The Fog Node (or Cloud server).
- **The Weight Limit:** The Fog Node's strict physical limits (e.g., available RAM, CPU capacity, or bandwidth).
- **The Items:** The incoming IoT task requests.
- **The Item Weight:** The memory or processing power required to execute that specific task.
- **The Item Value:** The benefit of offloading it (e.g., minimizing delay and saving the IoT device's battery).

The NP-Hard Difficulty Level

- **The Core Definition:** "NP-Hard" is a computer science classification that basically means: As the problem gets bigger, the number of possible combinations explodes so fast that it is impossible to calculate the perfect answer quickly.
- **The Small Scale:** If we only have 3 tasks to offload, it is easy. A computer can check all 2^3 x (8) possible routing combinations in a microsecond and pick the perfect one.
- **The Combinatorial Explosion:** If a smart city network has 1,000 tasks coming in at once, the system now has 2^1000 possible combinations to check. Even the most powerful supercomputer on Earth would take millions of years to calculate every single combination to find the mathematically perfect routing map.

metaheuristic Genetic Algorithms NSGA2, Bees Algorithm BA

is a higher-level algorithm that manages and guides a bunch of smaller heuristics to find a really great answer when we don't have the time or computing power to find the *perfect* answer. Imagine we are blindfolded and dropped into a massive mountain range. our goal is to find the absolute highest peak in the entire range. **The Metaheuristic Approach (Like NSGA-II or Bees Algorithm):** we drop 100 hikers randomly across the entire map. we tell them all to walk uphill. Every hour, we check in. we take the hikers stuck on small hills and teleport them near the hikers who are finding massive mountains. we also randomly drop a few new hikers in totally unexplored areas, just in case.You might not find the single highest rock on the absolute highest peak, but within a few minutes, you will find a mountain that is 99% as good.

The Core Pillars of Metaheuristics

Every metaheuristic algorithm (Genetic Algorithms, Bees Algorithm, Ant Colony, Simulated Annealing) is built on balancing two competing forces:

1. **Exploration (Casting a wide net):** Randomly searching completely new areas of the data to make sure you didn't miss a hidden goldmine.
2. **Exploitation (Drilling down):** Taking the good solutions you've already found and tweaking them slightly to squeeze out an even better score.

This exploration vs. exploitation trade-off is actually everywhere in computer science. If you were building a Reinforcement Learning model—like a Deep Q-Network (DQN) to optimize a news feed—you'd face the exact same dilemma: do you *exploit* the content you already know the user clicks on, or do you *explore* new content to see if it yields a higher reward? Metaheuristics apply that same logic to optimization puzzles.

In an IoT-Fog-Cloud architecture, balancing energy and delay across hundreds of devices creates that "NP-hard" combinatorial explosion. You can't measure the whole mountain range. Metaheuristics are the only way to quickly shuffle through millions of offloading combinations, dodge the bad ones, and spit out a highly optimized routing map in under 2 seconds.

NIP (Non-linear Integer Programming)

The concept: The "Brute-Force" Exact Solver. **How it works:** NIP doesn't guess. It uses strict mathematical equations to evaluate every single valid combination in the search space. If we have 10 IoT tasks and 3 Fog nodes, NIP calculates the exact energy and delay for every possible routing map, checks them against the memory constraints, and picks the undisputed, mathematically perfect winner.

Because task offloading is an NP-hard problem, the time complexity is roughly exponential, like O(m^n)

where m is the number of nodes and n is the number of tasks. For 5 tasks, it takes milliseconds. For 2,500 tasks, it would take huge amount of time

NSGA-II (Non-dominated Sorting Genetic Algorithm II)

The concept: The "Evolutionary Array" approach.

How it works: Instead of checking every combination, NSGA-II takes smart, evolutionary guesses.

- **The Chromosome (Array):** It generates a random 1D array where each index represents a task, and the value represents the node it gets sent to. It makes a "population" of, say, 100 of these random arrays.
- **Fitness:** It runs these 100 arrays through the energy and delay equations.
- **Evolution:** It takes the best-performing arrays, "breeds" them by swapping halves (Crossover), and randomly flips a few values (Mutation) to create a new, hopefully better, generation of 100 arrays.
- **Pareto Sorting:** Because we have *two* competing goals (Energy vs. Delay), there is no single "best" array. NSGA-II sorts the arrays into a "Pareto Front"—a curve of the absolute best trade-offs (e.g., one array gives the lowest possible energy for a 50ms delay, another gives the lowest energy for a 100ms delay).

Bees Algorithm (BA)

The concept: The "Swarm Search" approach (Exploration vs. Exploitation). **How it works:** This mimics how honey bees forage. It is a massive neighborhood search.

- **Scouts (Exploration):** The algorithm casts random "scout" solutions across the entire search space to evaluate random routing maps.
- **Workers (Exploitation):** When a scout finds a "patch" (a routing map) that yields a low energy/delay score, the algorithm sends a swarm of "worker bees" to that specific area. The workers essentially tweak the variables of that good solution slightly to see if they can find a local optimum right next to it.
- It's constantly balancing casting a wide net (scouts) while drilling down deeply into promising areas (workers).

MMDE (Minimax Differential Evolution)

The concept: The "Turbocharger" for metaheuristics. **How it works:** Standard genetic algorithms (like NSGA-II) waste a lot of time randomly mutating bad solutions. Sometimes a mutation accidentally makes the array worse, stalling the optimization process. MMDE fixes this by acting as an aggressive filter before the next generation is created.

- **Bottom-Boosting:** It identifies the worst-performing solutions in the current generation (the "bottom").
- Instead of randomly mutating them, it looks at the vector difference between the *best* solutions. It uses that mathematical difference to forcibly drag the bad solutions into a better neighborhood of the search space.
- By actively fixing the worst-case scenarios and refusing to let weak solutions breed, MMDE violently accelerates the convergence speed, dropping the time it takes to find the optimal offloading map down to under 2 seconds.

Documentation :

The paper tackles a critical bottleneck in modern computing: Internet of Things (IoT) devices generate massive amounts of data (from smart cities, VR/AR, vehicular networks) that require heavy processing. Sending all this data to a distant centralized Cloud causes unacceptable delays (latency) for time-sensitive applications. To fix this, "Fog computing" places processing nodes at the edge of the network, closer to the users.

However, **deciding exactly *where* to send a task (stay local, go to the Fog, or go to the Cloud) is highly complex. Offloading to the Fog saves the IoT device's battery but increases response time compared to local execution. Thus, a precise trade-off must be struck between Energy Consumption and Time Delay.**

used NSGA-II-MMDE
formulate the problem of joint optimization of energy consumption and latency in the form of a multi-objective problem and solve it using the non-dominant sorting genetic algorithm (NSGAII) and Bees algorithm (BA)
An evolutionary approach called minimax differential evolution (MMDE)

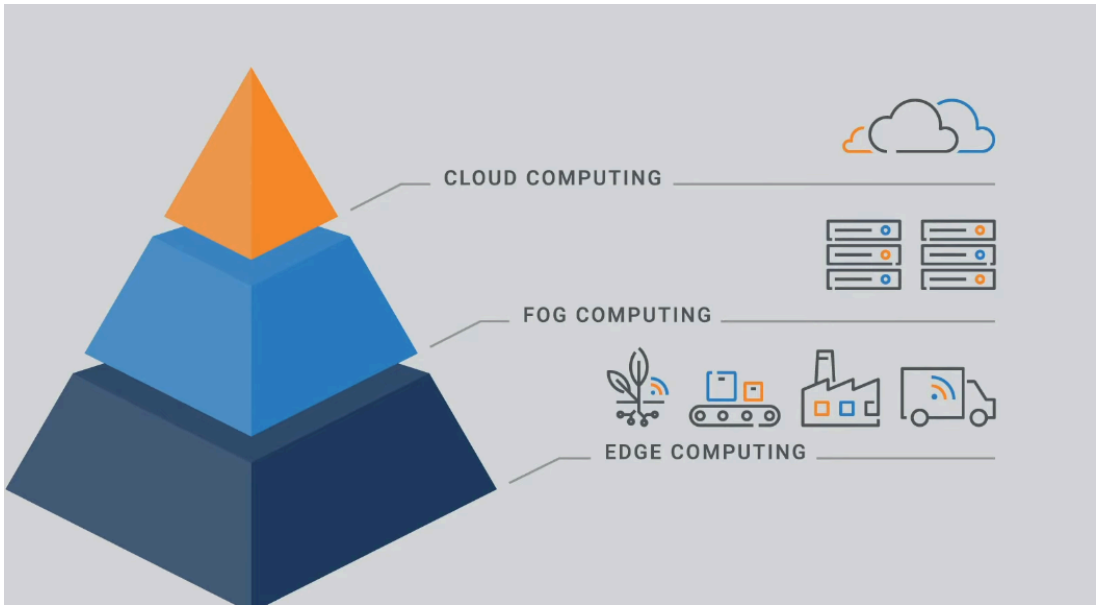
<u>System Architecture</u>

The authors model a three-tier environment:

- **Tier 1 (IoT Devices):** Generate tasks. If a device lacks resources, it offloads the task.
- **Tier 2 (Fog Instances):** Edge nodes accessed via Base Stations (BS). They have limited processing power and memory but offer low latency.
- **Tier 3 (Cloud Datacenter):** Massive processing power, but high transmission latency.

The system allows three offloading pathways:

- IoT-to-Fog,
- Fog-to-Cloud (if the Fog gets overwhelmed),
- direct IoT-to-Cloud.



Mathematical Modeling (The Objective Functions)

The authors mathematically model the cost of every decision.

- **Energy Model:** The total energy E is the sum of the energy consumed across all requests. This includes the transmission energy (through base stations and routers), processing energy, storage energy, and the energy required to send the response back.
- **Delay Model:** The total delay T includes transmission latency, processing latency at the node, and the time it takes to send the result back.
- **Constraints:** An offloaded task must not exceed the available memory of the target node, and the total processing time must not exceed the task's strict deadline.

The Optimization Algorithms

Because task offloading is an NP-hard knapsack problem, traditional math cannot solve it quickly at scale. The authors use three approaches:

- **NIP (Non-linear Integer Programming):** Used as the deterministic baseline to find the absolute "perfect" global optimal solution. However, its time complexity explodes as devices increase, making it useless for real-world application.
- **NSGA-II (Non-dominated Sorting Genetic Algorithm II):** A genetic algorithm that represents offloading decisions as "chromosomes." It sorts solutions into Pareto fronts to find the best multi-objective trade-offs.

- **Bees Algorithm (BA):** Another swarm-based metaheuristic algorithm tested for comparison.

When wehave 2,500 different IoT devices all firing off tasks at the same time, the central system has to decide: *"Which task goes to Fog Node A, which goes to Fog Node B, and which gets sent all the way to the Cloud?"*
Because the system cannot exceed the memory limits of any single Fog Node (the backpack weight limit), it has to test millions of combinations to find the exact routing map that saves the most energy while meeting every deadline.
This is exactly why the authors' strictly mathematical method (NIP) choked during the simulation. It tried to calculate every combination perfectly. Instead, they had to use the **NSGA-II metaheuristic algorithm** because genetic algorithms don't look for the "perfect" answer—they take smart guesses and evolve them to find a "really good" answer in under 2 seconds.

MMDE Evolution

To improve the metaheuristic algorithms, the authors hybridized them with **Minimax Differential Evolution (MMDE)**. MMDE uses a "bottom-boosting" strategy and partial-regeneration. Instead of wasting computational time evaluating weak, worst-case offloading scenarios, MMDE aggressively pushes the genetic mutations toward better optimization areas, severely increasing convergence speed.

5. Simulation and Results

The authors simulated the environment using **iFogSim**.

- **Energy & Delay:**
  - The default First-Come-First-Serve (FCFS) routing was terrible. NIP was mathematically perfect.
  - The **NSGA-II-MMDE** hybrid performed exceptionally well, achieving energy and delay metrics almost identical to the perfect NIP baseline.
- **Speed:**
  - While NIP took massive amounts of time to compute the routes, the NSGA-II-MMDE found the optimal offloading assignments in under 2 seconds, making it viable for dynamic networks.
  - BA-based methods were noticeably slower and less stable (higher variance).